

ZEGOWSKA SZAMA

Oficjalna Dokumentacja Projektu

Zespół programistyczny: Filip Hyski, Krzysztof Panek

Czas realizacji projektu: od 23.03.2026 do 30.05.2026

Strona projektu: <https://zegowska-szama.page.gd>

Ogólny opis projektu

„Zegowska Szama” to prosta aplikacja e-commerce napisana w PHP, obsługująca przeglądanie produktów, rejestrację i logowanie użytkowników, zarządzanie ofertami oraz realizację zamówień.

Aplikacja udostępnia interfejs webowy (HTML/CSS/JS) oraz warstwę serwerową w PHP z zapisem w relacyjnej bazie danych (MySQL/SQLite).

Zlecenie użytkownika

Uczniowie, nauczyciele oraz zespół prowadzący szkolny sklepik „Zegowska Szama” chcą korzystać z nowoczesnej aplikacji webowej, która:

- umożliwia przeglądanie oferty i zamawianie produktów online,
- pozwala zespołowi sklepu zarządzać użytkownikami, promocjami i przyjmować zamówienia,
- umożliwia wysyłanie istotnych komunikatów dla prawidłowego funkcjonowania sklepu.

Wymagania funkcjonalne

- Rejestracja konta użytkownika
- Logowanie / wylogowanie użytkownika
- Przeglądanie listy produktów
- Filtry i wyszukiwanie produktów
- Dodawanie produktu do koszyka i składanie zamówienia
- Zarządzanie ofertami (dodawanie/edycja/usuwanie) dla administratora
- Obsługa promocji i kodów rabatowych
- Wyświetlanie historii zamówień
- Powiadomienia o stanie zamówienia

Wymagania niefunkcjonalne

- **Wydajność:** obsługa kilku równoczesnych użytkowników bez zauważalnych opóźnień przy typowym ruchu

- **Bezpieczeństwo:** hasła przechowywane w formie zahashowanej, ochrona przed SQL Injection
- **Skalowalność:** struktura kodu umożliwiająca rozszerzanie modułów
- **Dostępność:** responsywny interfejs działający na desktopie i mobilnie
- **Utrzymywalność:** czytelna struktura plików i modularny kod

Opis techniczny

Architektura systemu

Aplikacja jest zorganizowana wokół uproszczonego wzorca podobnego do MVC:

- **Model:** obsługa danych i dostęp do bazy. W tej warstwie znajduje się `db.php` oraz skrypty SQL w plikach `.sql` odpowiedzialne za zapytania, relacje i zapisywanie/odczyt danych.
- **Widok:** generowanie interfejsu użytkownika. Widok realizują pliki PHP zwracające HTML oraz skrypty JS, które dynamicznie renderują dane i obsługują zachowanie strony. Przykłady: `index.php`, `offers.php`, `products-render.js`.
- **Kontroler:** logika aplikacji i przetwarzanie żądań. Kontrolerami są skrypty, które odbierają parametry HTTP, walidują dane, wykonują operacje na modelu i zwracają odpowiednie widoki lub JSON. Przykłady: `offers-api.php`, `checkout.php`, `manage.php`.

Ta struktura pozwala na oddzielenie dostępu do danych od logiki aplikacji i warstwy prezentacji, choć wszystkie komponenty znajdują się w jednym katalogu projektu.

Schemat bazy danych (ERD) — opis i tabele

Poniżej znajduje się szczegółowy opis struktur tabel, kluczy, indeksów i ograniczeń integralnościowych zaimplementowanych w relacyjnej bazie danych na podstawie technicznej dokumentacji systemu.

Wykaz i specyfikacja tabel

- **products** (Dostępne produkty)
 - `id` : integer, klucz główny (PK), auto-inkrementacja.
 - `name` : string – nazwa produktu.
 - `category` : string – kategoria przynależności.
 - `picture` : string, unikalna ścieżka do zdjęcia produktu.
 - `price_cents` : integer – cena produktu przechowywana w groszach (`price_cents > 0`). Wartość tylko do odczytu (READ-ONLY) z poziomu logiki biznesowej.
 - `discount_percent` : integer, dopuszcza wartości NULL – procentowy rabat na produkt (wartości ograniczone regułą `BETWEEN 0 AND 100`).
 - `stock` : integer – aktualny stan magazynowy (`stock >= 0`).

- **orders** (Utworzone zamówienia)
 - **id** : integer, klucz główny (PK), auto-inkrementacja.
 - **user_id** : integer, klucz obcy (FK) wskazujący na tabelę **users** (pole indeksowane).
 - **status** : integer – reprezentacja stanu zamówienia przyjmująca wartości ograniczone regułą **IN (0, 1, 2, 3)** (gdzie: **0** - oczekujące / pending, **1** - w trakcie przetwarzania / processing, **2** - ukończone / completed, **3** - opłacone i odebrane / paid & collected).
 - **created_at** : datetime – data i godzina złożenia zamówienia.
- **order_items** (Pozycje zamówień – tabela migawkowa/snapshot)
 - **id** : integer, klucz główny (PK), auto-inkrementacja.
 - **order_id** : integer, klucz obcy (FK) wskazujący na tabelę **orders** (pole indeksowane).
 - **product_id** : integer, klucz obcy (FK) wskazujący na tabelę **products** (pole indeksowane).
 - **quantity** : integer – zamówiona liczba sztuk (**quantity > 0**).
 - **unit_price_snapshot** : integer – cena jednostkowa produktu w groszach z momentu zakupu (**unit_price_snapshot > 0**).
 - **discount_percent_snapshot** : integer, dopuszcza wartości NULL – wartość rabatu procentowego zamrożona w momencie zakupu.
- **users** (Zarejestrowani użytkownicy)
 - **id** : integer, klucz główny (PK), auto-inkrementacja.
 - **name** : string – nazwa użytkownika.
 - **password** : string – zahashowane hasło (pole indeksowane).
 - **email** : string, unikalny – adres poczty elektronicznej.
 - **role_id** : integer, klucz obcy (FK) wskazujący na tabelę **roles** .
 - **created_at** : datetime – data i godzina rejestracji konta.
- **roles** (Dostępne role systemowe)
 - **id** : integer, klucz główny (PK), auto-inkrementacja.
 - **name** : string – nazwa roli (np. admin, user).
- **permissions** (Słownik uprawnień)
 - **id** : integer, klucz główny (PK), auto-inkrementacja.
 - **name** : string – nazwa uprawnienia. Zdefiniowane poziomy bitowe/wartości mapowania:
 - **0** – przeglądanie ofert (see offers)
 - **1** – zarządzanie zamówieniami (manage orders)
 - **2** – publikacja ogłoszeń (push announcements)
 - **3** – zarządzanie rolami (manage roles)
- **role_permissions** (Tabela pośrednicząca mapowania uprawnień do ról)
 - **role_id** : integer, klucz obcy (FK) wskazujący na tabelę **roles** .

- `permission_id` : integer, klucz obcy (FK) wskazujący na tabelę `permissions`.
- **Klucz główny (PK)**: złożony z pól (`role_id` , `permission_id`).
- **settings** (Indywidualne ustawienia użytkownika)
 - `user_id` : integer, klucz główny (PK), będący jednocześnie kluczem obcym (FK) wskazującym na tabelę `users`.
 - `theme` : string – preferowany motyw graficzny interfejsu (np. jasny/ciemny).
- **offers** (Metadane pakietów promocyjnych / zestawów ofertowych)
 - `id` : integer, klucz główny (PK), auto-inkrementacja.
 - `name` : string – nazwa zestawu promocyjnego.
 - `price` : integer – całkowita cena specjalna pakietu wyrażona w groszach.
 - `created_at` : datetime – data utworzenia pakietu promocyjnego.
- **offer_products** (Tabela łącząca – produkty wchodzące w skład oferty pakietowej)
 - `offer_id` : integer, klucz obcy (FK) wskazujący na tabelę `offers` (pole indeksowane).
 - `product_id` : integer, klucz obcy (FK) wskazujący na tabelę `products` (pole indeksowane).
 - `quantity` : integer – liczba sztuk danego produktu wchodząca w skład pakietu (`quantity > 0`).
 - **Klucz główny (PK)**: złożony z pól (`offer_id` , `product_id`).
- **order_offers** (Rejestr pakietów zakupionych w ramach zamówień – tabela migawkowa/snapshot)
 - `id` : integer, klucz główny (PK), auto-inkrementacja.
 - `order_id` : integer, klucz obcy (FK) wskazujący na tabelę `orders` (pole indeksowane).
 - `offer_id` : integer, klucz obcy (FK) wskazujący na tabelę `offers` (pole indeksowane).
 - `quantity` : integer – liczba zakupionych zestawów ofertowych (`quantity > 0`).
 - `unit_price_snapshot` : integer – cena pakietu w groszach zamrożona w momencie zakupu (`unit_price_snapshot > 0`).
- **announcements** (Ogłoszenia i komunikaty systemowe platformy)
 - `id` : integer, klucz główny (PK), auto-inkrementacja.
 - `title` : string – tytuł komunikatu.
 - `content` : string – treść ogłoszenia.
 - `created_at` : datetime – data i godzina publikacji.

Relacje w bazie danych

Konfiguracja powiązań między encjami w schemacie relacyjnym prezentuje się następująco:

- `products` ↔ `orders` — Wiele do wielu (poprzez tabelę pośredniczącą `order_items`).
- `users` → `orders` — Jeden do wielu (użytkownik może złożyć wiele zamówień).
- `settings` ↔ `users` — Jeden do jednego (każdy użytkownik ma jeden rekord konfiguracji).

- `users` → `roles` — Jeden do wielu (rola grupuje wielu użytkowników).
- `roles` ↔ `permissions` — Wiele do wielu (poprzez tabelę mapującą `role_permissions`).
- `products` ↔ `offers` — Wiele do wielu (poprzez tabelę składową `offer_products`).
- `users` ↔ `offers` — Wiele do wielu.
- `orders` ↔ `offers` — Wiele do wielu (poprzez tabelę pozycji pakietowych `order_offers`).

API / komunikacja i format danych

Komunikacja wewnętrzna odbywa się przez standardowe żądania HTTP POST/GET do endpointów PHP (np. `offers-api.php`, `offers-api.php?action=...`).

Format danych: JSON dla odpowiedzi API (np. listy ofert, statusu akcji), HTML dla stron renderowanych po stronie serwera.

Przykład żądania AJAX (JS):

```
fetch('offers-api.php', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({action: 'list', filter: {}})
}).then(r => r.json()).then(data => console.log(data));
```

Instalacja i wdrożenie

Wymagania hostingowe

- Serwer WWW: Apache lub Nginx
- PHP 7.4+ z rozszerzeniami `pdo_mysql`, `mysqli`, `mbstring`, `json`, `gd` (jeśli obsługa obrazów)
- MySQL 5.7+ lub MariaDB
- Dostęp do zapisu w katalogu `src/photos/` dla uploadów zdjęć

Instrukcja instalacji (lokalnie)

1. Sklonuj repozytorium.
2. Skopiuj plik konfiguracji bazy (jeśli istnieje) lub utwórz `db.php`, `checkout.php`, `manage.php` z połączeniem do DB.
3. Zaimportuj schemat bazy danych:

```
mysql -u user -p database_name < szama.sql
```

4. Umieść pliki na serwerze WWW (np. w katalogu `public_html` lub `www`).
5. Skonfiguruj uprawnienia katalogu `photos/` (np. `chmod 755 photos` lub `www-data` ownership).
6. Otwórz stronę w przeglądarce i przejdź przez proces rejestracji/testów.

Instrukcja wdrożenia na zewnętrznym hostingu

1. Wybierz hosting z obsługą PHP i MySQL (np. shared hosting, VPS).
2. Wgraj pliki projektu przez FTP/SFTP lub Git.
3. Stwórz bazę danych i użytkownika w panelu hostingu.
4. Zaimportuj plik `szama.sql` przez phpMyAdmin lub z linii poleceń.
5. Ustaw konfigurację połączenia do bazy w `db.php` i kilku innych plikach (`checkout.php` , `manage.php`).
6. Skonfiguruj domenę i upewnij się, że katalog główny wskazuje na miejsce z plikami projektu.

Instrukcja obsługi aplikacji (użytkownik)

• Rejestracja konta:

1. Wejdź na stronę rejestracji (`register.php`).
2. Wypełnij formularz: nazwa, email, hasło.
3. Zatwierdź. Po rejestracji możesz się zalogować.

• Logowanie:

1. Przejdź do `login.php` .
2. Wprowadź email i hasło.
3. Po zalogowaniu zostaniesz przekierowany do strony głównej.

• Składanie zamówienia:

1. Dodaj produkty do koszyka.
2. Przejdź do `checkout.php` .
3. Wypełnij dane wysyłki i potwierdź zamówienie.

• Korzystanie z promocji:

1. Na etapie checkout wprowadź kod promocji (jeśli mechanizm dostępny).
2. System zastosuje rabat zgodnie z regułami oferty.

• Zarządzanie ofertami (admin):

1. Zaloguj się jako administrator.
2. Przejdź do `manage.php` .
3. Dodaj/edytuj/usun oferty i produkty.

Załączniki i pliki źródłowe

- Schemat bazy danych i dane przykładowe: `docs/szama.sql` .
- Projekty graficzne i diagramy: katalog `design/` .
- Pliki projektu: katalog `src/` .
- Katalog zdjęć produktów `src/photos` .